UE24CS242B: Operating Systems – Orange Problem (Unit 4)

Name: Nihar Sureshwar Oak

Section: E

SRN: PES1UG24CS299

Phase 1:

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2:

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ find .pes/objects -type f
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ xxd .pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f | head -20
00000000: 626c 6f62 2036 0077 6f72 6c64 0a         blob 6.world.
```

Phase 3:

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ echo "hello" > file1.txt
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ echo "world" > file2.txt
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ ./pes add file1.txt file2.txt
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  test_objects
  untracked:  README.md
  untracked:  commit.c
  untracked:  pes.c
  untracked:  test_sequence.sh
  untracked:  tree.h
  untracked:  commit.h
  untracked:  index.h
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  index.c
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  object.c
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776787308 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776787316 6 file2.txt
```

Phase 4:

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ ./pes log
commit 0d76c90eaadede20a20031a342d599018dafcf307c452480c61ae2933e3fa8d9
Author: Nihar <PES1UG24CS299>
Date:   1776795074

    Add farewell


commit 2fa96d37a2682cf5acc553196aeea4d133908982900a2fb9919eec26804b4c13
Author: Nihar <PES1UG24CS299>
Date:   1776795031

    Add world


commit 64ac22385a851d4ef9dd69812fcd5ef85dc39ef32d17f1ed310b22dff0f462f8
Author: Nihar <PES1UG24CS299>
Date:   1776794999

    Initial commit
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0d/76c90eaadede20a20031a342d599018dafcf307c452480c61ae2933e3fa8d9
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/2f/a96d37a2682cf5acc553196aeea4d133908982900a2fb9919eec26804b4c13
.pes/objects/5d/343ad75278b7bb7e4deac2e7c5c5bd7b20893e8d659ac5c601b2ec927145c9
.pes/objects/64/ac22385a851d4ef9dd69812fcd5ef85dc39ef32d17f1ed310b22dff0f462f8
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/71/7a6e47adf1724af06cec666a67cd41047178120ce1a24a7c40549e735aae3c
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/c9/6f2fc25394979be592d5cf80c5d78ec792ff13dde17f838449fbcdfbb7df0d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f0/1d8c6928a88e5f110760f6fc003facb8af3324620d04792df75b18bc8dc274
.pes/refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ cat .pes/refs/heads/main
0d76c90eaadede20a20031a342d599018dafcf307c452480c61ae2933e3fa8d9
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final: Full integration test (make test-integration)

```
nihar@nihar-VirtualBox:~/PES1UG24CS299-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: eae0dee21967... Initial commit

Log after first commit:
commit eae0dee219671bd052ccb519e82de2339fe347cb93ff047f6204993b2c79e994
Author: Nihar <PES1UG24CS299>
Date:   1776795587

    Initial commit



--- Second Commit ---
Committed: 7981487ed312... Update file.txt

--- Third Commit ---
Committed: 1421af4c9b03... Add farewell

--- Full History ---
```

Analysis only questions (Phases 5&6):

Q5.1 — Implementing pes checkout <branch>

To implement pes checkout <branch>, three things must happen:

Files that change in .pes/:

- HEAD is rewritten to contain ref: refs/heads/<branch>

Working directory update:

1. Read the target branch's commit hash from .pes/refs/heads/<branch>
2. Walk that commit's tree recursively through the object store
3. For every blob in the target tree, read its contents and overwrite the corresponding working directory file
4. Delete any files tracked in the current index that don't exist in the target tree
5. Replace the index entirely with entries reflecting the target tree's files

What makes it complex:

- Files present in the current branch but absent in the target must be deleted — requiring a diff between two trees
- New directories may need to be created, and empty directories left behind must be cleaned up
- The operation must be atomic enough that a crash midway doesn't leave the repo in a broken half-switched state
- Untracked files that would be overwritten by the checkout must be detected and the operation aborted to avoid data loss

Q5.2 — Detecting a Dirty Working Directory Conflict

When switching branches, a conflict exists when a file satisfies all three conditions simultaneously:

1. **It exists in the current index** (it is tracked)
2. **It has been modified in the working directory** — detected by comparing mtime and size from the index entry against the live stat() of the file (the same fast-diff used in index_status)
3. **It differs between the two branches' trees** — detected by comparing the blob hash stored in the current index against the blob hash for that path in the target branch's tree object

If conditions 1 and 2 are true but condition 3 is false (the file is the same in both branches), checkout can safely proceed — the working directory change would be preserved. Only when all three are true must checkout refuse, because it would need to overwrite a locally modified file with different content from the target branch.

No re-hashing of file contents is needed — the mtime/size fast-diff from the index is sufficient for detection.

Q5.3 — Detached HEAD

In detached HEAD state, HEAD contains a raw commit hash directly instead of ref: refs/heads/<branch>. New commits are still created normally — each one points to the previous as its parent. However, no branch file is updated, so the only pointer to these commits is HEAD itself.

If you then run checkout <branch>, HEAD gets overwritten with the new branch reference, and the chain of commits made in detached state becomes **unreachable** — no branch, tag, or ref points to them. They aren't immediately deleted but will be collected by garbage collection eventually.

Recovery options:

- If you remember the commit hash (visible in the terminal history), create a new branch pointing to it: pes branch recovery-branch <hash>, which writes that hash to .pes/refs/heads/recovery-branch
- In real Git, git reflog records every position HEAD has pointed to, making recovery straightforward even without remembering the hash

Q6.1 — Garbage Collection Algorithm

Algorithm (mark-and-sweep):

1. **Mark phase** — collect all reachable object hashes into a hash set:
    - Read every file in .pes/refs/heads/ to get all branch tip commit hashes
    - For each commit hash, read the commit object and add it to the reachable set
    - Follow the parent pointer and repeat until reaching the root commit
    - For each commit, add its tree hash; recursively walk the tree, adding every subtree and blob hash encountered
2. **Sweep phase** — walk every file under .pes/objects/XX/YYY..., reconstruct the full hash from the directory name + filename, and delete any file whose hash is not in the reachable set

Data structure: A hash set (open-addressing or chained) keyed on the 32-byte ObjectID gives O(1) lookup during the sweep phase.

Estimate for 100,000 commits, 50 branches: Assuming an average of 10 objects per commit (1 commit + 1–2 trees + 7–8 blobs), you'd visit roughly **1,000,000 objects** during the mark phase. The sweep phase visits every file in the object store once. Both phases are linear in the number of objects.

Q6.2 — GC Race Condition with Concurrent Commit

The race:

1. A commit operation calls tree_from_index, which writes several new blob and tree objects to the store — but has not yet called head_update
2. GC starts its mark phase, reads all branch refs, and builds the reachable set — the new objects are not yet reachable because HEAD still points to the old commit
3. GC's sweep phase deletes the newly written blobs and trees as "unreachable"
4. The commit operation calls object_write for the commit object and then head_update — but the tree it references has just been deleted, leaving the repository corrupt

How real Git avoids this: Git uses a **grace period** — GC never deletes any object written more recently than 14 minutes ago (configurable via gc.pruneExpire), regardless of reachability. Since a commit operation completes in milliseconds, any objects written during an ongoing commit are protected by this window. This avoids the need for any locking between GC and writers.